MongoDB | Class Notes

Nested Documents · Arrays · Validation · Query Operators · Null vs Missing
Quick reference. Each topic: syntax, what it does, an example.

0. Sample Data (run this first)

Most examples use the `students` collection below. A few use a separate `employees_validated` collection (shown in the Validation section).

```
use school

db.students.insertMany([
  { name:'Pooja', age:22, city:'Delhi',  course:'MCA',   minAge:18,
    subjects:['Math','Science','English'],
    marks:[ {subject:'Math',score:90}, {subject:'Science',score:85} ] },

  { name:'Aman',  age:24, city:'Mumbai', course:'BCA',   minAge:20,
    subjects:['Math','English'],
    marks:[ {subject:'Math',score:70}, {subject:'Science',score:60} ] },

  { name:'Riya',  age:18, city:'Delhi',  course:'BTech', minAge:19,
    subjects:['Science','English'],
    marks:[ {subject:'Science',score:95}, {subject:'Math',score:50} ] },

  { name:'Rahul', age:22, city:'Noida',  course:'MCA',   minAge:25,
    subjects:['Math','Science'],
    marks:[ {subject:'Math',score:88}, {subject:'Science',score:72} ] },

  { name:'Priya', age:26, city:'Pune',   course:'BCA',   minAge:21,
    subjects:['Math','Science','English'],
    marks:[ {subject:'Math',score:91}, {subject:'Science',score:80} ] },

  { name:'Anita', age:19, city:'Delhi',  course:'BTech', minAge:18,
    subjects:['English'],
    marks:[ {subject:'English',score:65} ] },

  { name:'Karan', age:28, city:'Mumbai', course:'MCA',   minAge:22,
    subjects:['Math','Science','English'],
    marks:[ {subject:'Math',score:78}, {subject:'Science',score:88} ] },

  { name:'Meera', age:null, city:'Pune', course:'BCA',   minAge:20,   // age is NULL
    subjects:['Science'],
    marks:[ {subject:'Science',score:70} ] },

  { name:'Sahil', city:'Noida', course:'BTech', minAge:19,            // age
MISSING
    subjects:['Math'],
    marks:[ {subject:'Math',score:60} ] },
```

```
  { name:'Divya', age:23, city:'Delhi',  course:'MCA',   minAge:20,
    subjects:['Math','English'],
    marks:[ {subject:'Math',score:82}, {subject:'English',score:90} ] }
])
```

Note: Meera's age is null (field present, no value). Sahil has no age field at all (missing). These two power the Null vs Missing section. Every student has a subjects array and a marks array-of-objects.

1. Nested Documents (objects inside a document)

What it is: A field whose value is itself an object with its own key:value pairs. Keeps related data (like an address) inside the same document instead of a separate table.

Insert

Syntax: **`db.<col>.insertOne({ field: { key: value, ... } })`**

```
db.students.insertOne({
  name: 'Vihaan',
  age: 29,
  address: {                    // nested document (object)
    city: 'Bangalore',
    state: 'Karnataka',
    zipCode: '560001'
  }
})
```

Query (dot notation — path MUST be in quotes)

Syntax: **`db.<col>.find({ 'parent.child': value })`**

```
db.students.find({ 'address.city': 'Bangalore' })

// multiple nested conditions
db.students.find({ 'address.state':'Karnataka', 'address.zipCode':'560001' })
```

Quote warning: The dotted path must be quoted — 'address.city'. Writing { address.city: ... } without quotes is a JavaScript syntax error.

2. Arrays (lists inside a document)

What it is: A field holding a list of values — skills, tags, etc. Use when a field naturally has more than one value.

Insert

Syntax: **`db.<col>.insertOne({ field: [ val1, val2, ... ] })`**

```
db.students.insertOne({
  name: 'Ishaan',
  skills: ['JavaScript', 'React', 'MongoDB']   // array
})
```

Query (just the field name — no dot, no quotes)

Syntax: **`db.<col>.find({ arrayField: value })`**

MongoDB checks every element. The document matches if ANY element equals the value.

```
db.students.find({ subjects: 'Math' })   // anyone whose array contains Math
```

$in (match any of several values)

Syntax: **`db.<col>.find({ arrayField: { $in: [val1, val2] } })`**

```
db.students.find({ subjects: { $in: ['Math','Science'] } })
// matches if the array contains Math OR Science
```

Key difference: Nested object → quoted dot path. Array → plain field name, matches any element.

3. Nested Document vs Array (querying)

Question	Nested Document	Array
Example data	`address: { city:'Delhi' }`	`skills: ['JS','MongoDB']`
Query syntax	`{ 'address.city':'Delhi' }`	`{ skills:'MongoDB' }`
Needs quotes?	YES — quote the dot path	NO quotes
Uses dot notation?	YES	NO
What it matches	Exact nested field value	Any element in the array

4. Schema Validation — Why

Problem: MongoDB is schema-less, so any document can have any field/type. In a team this leads to bad data — age stored as a string, missing required fields, negative values, inconsistent data that is hard to debug later.

Solution: Add optional validation rules to a collection. Documents that break the rules are rejected — on BOTH inserts and updates. You stay flexible but strict where it matters.

5. Validator structure

Validation is added when you CREATE the collection, using `$jsonSchema`.

Syntax: **`db.createCollection('<name>', { validator: { $jsonSchema: {...} } })`**

```
db.createCollection('employees_validated', {
  validator: {
    $jsonSchema: {
    bsonType: 'object',              // the document is an object
    required: ['name', 'age'],       // these fields MUST exist
    properties: {
    name: {
           bsonType: 'string',
           description: 'must be a string'
    },
    age: {
           bsonType: 'int',
           minimum: 18,
           description: 'must be an int >= 18'
    }
    }
    }
  }
})
```

What each part does

- `validator` — wrapper that holds the rules.
- `$jsonSchema` — the rule language.
- `bsonType:'object'` — the document itself is an object (always start here).
- `required:[...]` — array of field names that must be present in every document.
- `properties:{...}` — per-field rules (type, min/max, etc.).
- `bsonType` (inside a field) — the type that field must be: string, int, double, bool, array, object, date.
- `minimum / maximum` — numeric range constraints.

6. Validation in action

REJECTED — three ways it fails

```
// 1) Missing required field
db.employees_validated.insertOne({ name: 'Tarun' })
//  -> fails: 'age' is required and missing
```

```
// 2) Wrong type
db.employees_validated.insertOne({ name: 'Riya', age: '25' })
//  -> fails: age specified as 'int', found 'string'

// 3) Constraint violation (below minimum)
db.employees_validated.insertOne({ name: 'Mini', age: 15 })
//  -> fails: age below minimum (18)

// All three give: MongoServerError: Document failed validation
```

ACCEPTED — valid data passes

```
db.employees_validated.insertOne({
  name: 'Pooja',  // string  OK
  age: 25,        // int >= 18  OK
  city: 'Delhi',  // extra fields are allowed (not validated)
  role: 'Developer'
})
//  -> acknowledged: true, insertedId: ObjectId(...)
```

Remember: Validation also applies to updates. Extra fields not named in the validator are still allowed — validation catches BAD data, it does not limit which fields you may add.

7. CRUD in MongoDB Compass (the visual way)

Compass is the GUI — do all CRUD by clicking, no commands.

Operation	How (in Compass)
Create DB / Collection	Plus icon next to the connection -> enter DB name + first collection name
Add collection	Plus icon next to an existing database
`Create (insert)`	ADD DATA -> Insert Document -> type JSON
`Read (query)`	Type a filter in the query bar -> FIND
Update	`Pencil icon -> double-click a value -> UPDATE (can also change type)`
Delete document	Trash icon on the document -> Delete
Drop collection	`Three dots -> Drop Collection (retype name)`
Drop database	`Trash icon on the database (retype name)`

Insert editor — quote the KEY too

```
{
```

```
  "name": "Pooja",       // both KEY and value are quoted here
  "age": 22,
  "city": "Delhi"
}
```

Common error: Forgetting quotes on the key gives 'Insert not permitted while document contains errors' — the Insert button stays disabled until the JSON is valid.

8. Logical Operators: $and / $or

$and — ALL conditions must be true. **$or** — at least ONE must be true. Both take an array of condition objects.

$and

Syntax: `{ $and: [ {cond1}, {cond2}, ... ] }`

```
// students in Delhi AND aged 20 or above
db.students.find({ $and: [ { city:'Delhi' }, { age:{ $gte:20 } } ] })
// -> Pooja (22), Divya (23). Not Riya/Anita (under 20).
```

Tip: For simple different-field equality, an implicit AND works too: { city:'Delhi', age:{$gte:20} }. Explicit $and is needed mainly when two conditions target the SAME field.

$or

Syntax: `{ $or: [ {cond1}, {cond2}, ... ] }`

```
// students in Mumbai OR doing MCA
db.students.find({ $or: [ { city:'Mumbai' }, { course:'MCA' } ] })
// a doc matching both still appears only once
```

9. Logical Operators: $not / $nor

$not — negates a single condition (wraps an operator INSIDE a field). **$nor** — NONE of the listed conditions are true (opposite of $or, takes an array).

$not

Syntax: `{ field: { $not: { operator } } }`

```
// city is NOT Delhi
db.students.find({ city: { $not: { $eq: 'Delhi' } } })
```

Note: $not { $eq:'Delhi' } is like $ne:'Delhi', but $not can wrap ANY operator, e.g. { age: { $not: { $gt: 20 } } }.

$nor

Syntax: `{ $nor: [ {cond1}, {cond2}, ... ] }`

```
// NOT in Delhi AND NOT aged 22
db.students.find({ $nor: [ { city:'Delhi' }, { age:22 } ] })
```

$or vs $nor: $or = include if at least one is true. $nor = include if none are true. Exact opposites.

10. $in / $nin

$in — field value is in the given list. **$nin** — field value is NOT in the list.

Syntax: `{ field: { $in: [a, b] } }` `{ field: { $nin: [a, b] } }`

```
// course is MCA or BCA
db.students.find({ course: { $in: ['MCA','BCA'] } })

// course is NOT MCA or BCA  -> only BTech students
db.students.find({ course: { $nin: ['MCA','BCA'] } })
```

Tip: $in replaces multiple $or conditions:

```
{ $or: [ {course:'MCA'}, {course:'BCA'} ] }   //  same as...
{ course: { $in: ['MCA','BCA'] } }           //  this (cleaner)
```

11. $expr — compare two fields in the same document

Normal query: compares a field to a FIXED value. **$expr:** compares a field to ANOTHER field in the same document.

Syntax: `{ $expr: { $gt: ['$fieldA', '$fieldB'] } }`

```
// students where minAge is greater than their actual age
db.students.find({ $expr: { $gt: ['$minAge', '$age'] } })
// -> Riya (minAge 19 > age 18), Rahul (minAge 25 > age 22)

// flip it: actual age greater than minAge
db.students.find({ $expr: { $gt: ['$age', '$minAge'] } })
```

$ prefix warning: Inside $expr, field names are strings with a $ in front: '$age', '$minAge'. The $ means 'this is a field reference, not a literal value'. Forgetting it is the most common $expr mistake.

12. Element Operators: $exists / $type

$exists — filter by whether a field is present. **$type** — filter by a field's data type. These match on STRUCTURE, not value.

$exists

Syntax: `{ field: { $exists: true | false } }`

```
db.students.find({ age: { $exists: true } })   // field present (incl. null!)
db.students.find({ age: { $exists: false } })  // field missing -> Sahil only
```

Note: $exists:true also includes documents where the field is null (like Meera). It checks presence, not value.

$type

Syntax: `{ field: { $type: 'typeName' } }    // or an array of type names`

```
db.students.find({ age: { $type: 'int' } })        // age stored as int
db.students.find({ age: { $type: ['int','double'] } }) // int OR double
```

Common BSON type names

Type	Means	Example
string	text	'Pooja'
int	32-bit whole number	22
long	64-bit whole number	large integers
double	decimal	88.5
bool	true / false	true
array	list	`['Math','Science']`
object	nested document	`{ city:'Delhi' }`
date	date value	`ISODate(...)`
null	null value	null

13. $regex — pattern matching (like SQL LIKE)

Searches text by pattern. Works on string fields.

Syntax: `{ field: { $regex: 'pattern' } }`

Pattern	Means	Example
^X	starts with X	`{ name: { $regex: '^P' } }`
x$	ends with X	`{ name: { $regex: 'a$' } }`

X	contains X anywhere	`{ name: { $regex: 'iy' } }`

```
db.students.find({ name: { $regex: '^P' } })   // starts with P -> Pooja, Priya
db.students.find({ name: { $regex: 'a$' } })   // ends with a -> Pooja, Riya, Priya, Anita, Divya
db.students.find({ name: { $regex: 'iy' } })   // contains iy -> Riya, Priya
```

Symbols: ^ = beginning of string, $ = end of string, no symbol = match anywhere.

14. $regex — case insensitive ($options: 'i')

By default regex is case-sensitive. Add `$options: 'i'` to ignore case.

Syntax: **`{ field: { $regex: 'pattern', $options: 'i' } }`**

```
// case sensitive — finds nothing (data has 'Riya' with capital R)
db.students.find({ name: { $regex: 'riya' } })

// case insensitive — matches 'Riya'
db.students.find({ name: { $regex: 'riya', $options: 'i' } })
```

Bonus: To match names containing any vowel: { name: { $regex: '[aeiou]' } } — square brackets mean 'any one of these characters'.

15. Array Queries: $all / $elemMatch

$all — array contains ALL listed values

Syntax: **`{ arrayField: { $all: [val1, val2] } }`**

```
// students whose subjects include BOTH Math AND Science
db.students.find({ subjects: { $all: ['Math','Science'] } })
// order doesn't matter; checks presence of every listed value
```

Query	Meaning
`{ subjects: 'Math' }`	`has Math (any single match)`
`{ subjects: { $all: ['Math','Science'] } }`	has BOTH Math AND Science
`{ subjects: { $in: ['Math','Science'] } }`	`has Math OR Science (at least one)`

$elemMatch — one array element must match ALL conditions

Syntax: `{ arrayField: { $elemMatch: { cond1, cond2 } } }`

Use it for arrays of OBJECTS (like `marks`), when several conditions must hold on the SAME element.

```
// marks looks like:
// marks: [ {subject:'Math',score:90}, {subject:'Science',score:85} ]

// students with a Science mark above 80
db.students.find({
  marks: { $elemMatch: { subject:'Science', score:{ $gt:80 } } }
})
// -> Pooja (Sci 85), Riya (Sci 95), Karan (Sci 88)
```

Why not plain dot notation?

```
// WRONG — conditions may match DIFFERENT elements
db.students.find({ 'marks.subject':'Science', 'marks.score':{ $gt:80 } })
// a doc with Math 90 + Science 60 would wrongly match

// CORRECT — both conditions on the SAME element
db.students.find({ marks: { $elemMatch: { subject:'Science', score:{ $gt:80 } } } })
```

Rule: Multiple conditions on the same array-of-objects element -> use $elemMatch.

16. Null vs Missing fields

Null: the field exists but its value is null (Meera: age:null). **Missing:** the field does not exist at all (Sahil: no age field).

The trap

Syntax: `{ field: null }   // matches null OR missing`

```
db.students.find({ age: null })
// returns BOTH Meera (null) AND Sahil (missing) — usually not what you want
```

Query precisely

Syntax: `missing only:  { field: { $exists: false } }`

Syntax: `null only:  { field: { $exists: true, $eq: null } }`

```
// missing only
db.students.find({ age: { $exists: false } })             // -> Sahil

// null only (field exists AND value is null)
db.students.find({ age: { $exists: true, $eq: null } })  // -> Meera

// field present with any value
db.students.find({ age: { $exists: true } })              // everyone except Sahil
```

Query	Returns
`{ age: null }`	`null OR missing (Meera + Sahil)`
`{ age: { $exists: false } }`	`missing only (Sahil)`
`{ age: { $exists: true, $eq: null } }`	`null only (Meera)`
`{ age: { $exists: true } }`	everyone except Sahil

End of notes — Nested · Arrays · Validation · Compass · Logical · $in/$nin/$expr · $exists/$type · $regex · $all/$elemMatch · Null vs Missing